

AI Learning Assistant: Emotion-Aware Learning Support Platform

Project Description:

This project presents an AI-powered learning assistant that detects emotions from students' free-text learning challenges and generates personalized guidance. The platform integrates NLP preprocessing, BiLSTM and BERT emotion classification, rule-based keyword enhancement, Gemini AI response generation, Streamlit-based interaction, Plotly analytics, and CSV logging.

The system transforms learner input into emotion-aware academic support. Students describe their learning difficulties, and the platform preprocesses the text, predicts emotions (Bored, Confident, Confused, Curious, Frustrated), estimates confidence, detects mixed emotions, and generates supportive responses using Gemini AI.

Problem Statement:

Students often struggle to express learning difficulties, and educators spend significant time manually interpreting student messages. Existing solutions generally provide generic responses without considering the learner's emotional state.

Objectives:

- Detect emotions using BiLSTM and BERT.
- Compare model predictions.
- Generate personalized guidance using Gemini AI.
- Visualize emotion trends with Plotly.
- Maintain interaction history using CSV logging.
- Provide a responsive Streamlit interface.

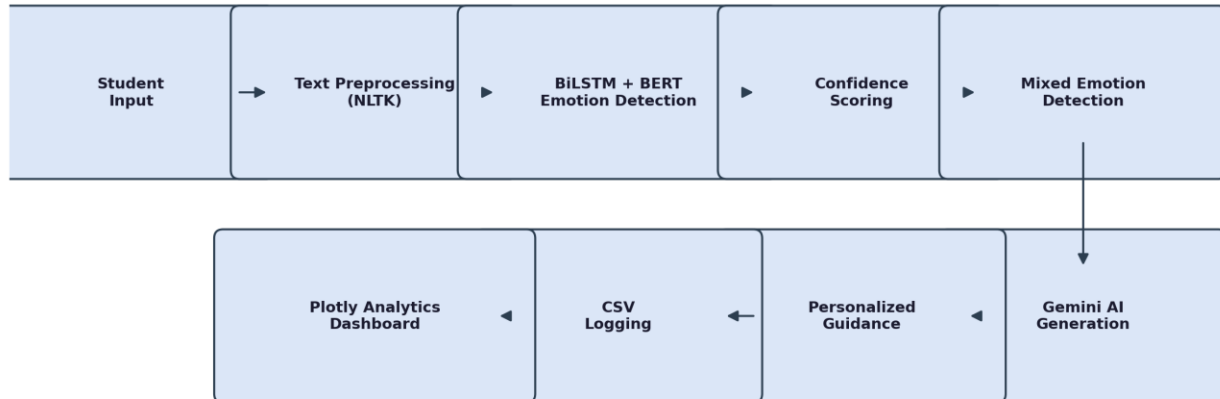
Scenarios

Scenario 1: Educators and academic support teams triage student concerns using automatic emotion detection.

Scenario 2: Self-study learners receive instant emotion-aware guidance.

Scenario 3: Institutions analyze emotion trends to improve learner support.

Technical Architecture:



Description: The AI Learning Assistant follows an end-to-end NLP and deep learning pipeline. Student input first passes through NLTK-based text preprocessing, after which BiLSTM and BERT models jointly classify the underlying emotion. Confidence scores are computed and mixed emotions are detected before Gemini AI generates a personalized, empathetic response. Each interaction is logged to CSV and visualized on a Plotly-powered analytics dashboard within the Streamlit interface.

Technology Stack:

Python, Streamlit, TensorFlow/Keras, PyTorch, Hugging Face Transformers, NLTK, NumPy, Pandas, Plotly, Scikit-learn, Gemini AI, Kaggle GPU.

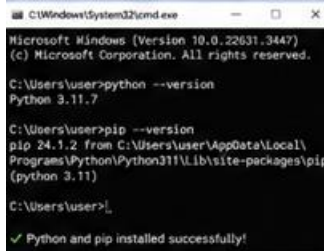
Pre-requisites:

- **Python Programming Proficiency:** docs.python.org/3/
- **Streamlit Framework Knowledge:** docs.streamlit.io/
- **Deep Learning with TensorFlow/Keras:** www.tensorflow.org/guide
- **Deep Learning with PyTorch:** pytorch.org/docs/stable/index.html
- **NLP & Transformers (Hugging Face):** huggingface.co/docs
- **Text Preprocessing with NLTK:** www.nltk.org/
- **Gemini AI API Familiarity:** ai.google.dev/docs
- **Data Visualization with Plotly:** plotly.com/python/
- **Development Environment Setup:** Python 3.8+, Jupyter Notebook, and VS Code or PyCharm.

Project Workflow:

Milestone 1: Environment Setup & Dependency Configuration

- **Activity 1.1:** Install Python 3.8+ and required system tools before beginning development.



```

C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.22631.3447]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>python --version
Python 3.11.7

C:\Users\user>pip --version
pip 24.1.2 from C:\Users\user\AppData\Local\Programs\Python\Python311\Lib\site-packages\pip
(python 3.11)

C:\Users\user>
  
```

✓ Python and pip installed successfully!

- **Activity 1.2:** Install core dependencies: Streamlit, TensorFlow/Keras, PyTorch, Hugging Face Transformers, NLTK, Plotly, Pandas, NumPy, and Scikit-learn.

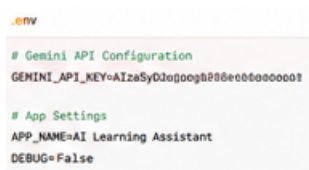


```

PS C:\AI_Learning_Assistant> pip install \
streamlit tensorflow keras torch \
transformers nltk plotly pandas numpy \
scikit-learn

Successfully installed:
  ✓ streamlit
  ✓ tensorflow
  ✓ keras
  ✓ torch
  ✓ transformers
  ✓ nltk
  ✓ plotly
  ✓ pandas
  ✓ numpy
  ✓ scikit-learn
  
```

- **Activity 1.3:** Configure the Gemini API key securely using a .env file.



```

.env

# Gemini API Configuration
GEMINI_API_KEY=AIzaSyD3oJ0o9th886e0b0sooooo0

# App Settings
APP_NAME=AI Learning Assistant
DEBUG=False
  
```

Milestone 2: Kaggle Model Training & Integration

- **Activity 2.1:** Perform preprocessing, tokenization, and label encoding on the emotion training dataset.



```

2_ Preprocessing.ipynb
1.: df.head()
  
```

	text	label
0	I don't understand this at all	confused
1	This is so boring and dull	bored
2	I am really frustrated right now	frustrated
3	This topic is so interesting!	curious
4	I feel confident about this	confident

```

✓ Lowercasing
✓ Removing special characters
✓ Tokenization (BERT Tokenizer)
✓ Label Encoding

Classes: ('bored', 'confident', 'confused', 'curious', 'frustrated')
  
```

- **Activity 2.2:** Train the BiLSTM and BERT models on Kaggle GPU resources.

```

3_Train_BiLSTM.ipynb

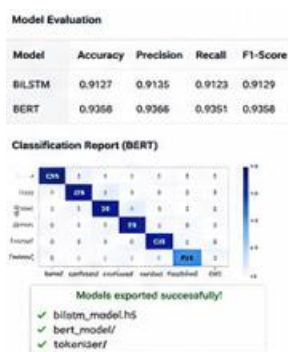
Epoch 1/10
100/100 [=====] - 12s 98ms/step -
loss: 0.7123 - accuracy: 0.7421
...
Epoch 10/10
100/100 [=====] - 9s 91ms/step -
loss: 0.2214 - accuracy: 0.9127

4_Train_BERT.ipynb

Epoch 1/3
100/100 [=====] - 35s 353ms/step -
loss: 0.5341 - accuracy: 0.9123
...
Epoch 3/3
100/100 [=====] - 31s 310ms/step -
loss: 0.1826 - accuracy: 0.9358

GPU: Tesla P100-PCIE-16GB
  
```

- **Activity 2.3:** Evaluate model performance and export the trained models for integration.



Milestone 3: Emotion Detection Pipeline

- **Activity 3.1:** Preprocess and tokenize incoming student text input.

```

Raw Input
I am so confused about recursion
and this is really frustrating!

Preprocessed Text
i am so confused about recursion
and this is really frustrating

Tokenized IDs (BERT)
[ 101 1045 2061 2004 3778 ... 102 ]

Attention Mask
[ 1 1 1 1 1 1 ... 1 ]

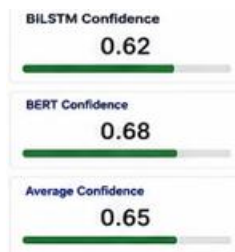
✓ Preprocessing & tokenization complete
  
```

- **Activity 3.2:** Classify emotions using both the BiLSTM and BERT models.

BiLSTM Prediction	
bored	0.05
confident	0.08
confused	0.62
curious	0.12
frustrated	0.13

BERT Prediction	
bored	0.04
confident	0.06
confused	0.68
curious	0.10
frustrated	0.12

- **Activity 3.3:** Compute confidence scores for each prediction.



- **Activity 3.4:** Detect mixed emotions and determine the final predicted emotion.



Milestone 4: AI-Powered Guidance (Gemini AI Integration)

- **Activity 4.1:** Design emotion-aware prompts using prompt engineering techniques.
- **Activity 4.2:** Generate empathetic explanations, personalized study strategies, motivation, and next steps via Gemini AI based on the detected emotion.

Milestone 5: Streamlit UI Development

- **Activity 5.1:** Build a problem input form for students to describe their learning challenges.
- **Activity 5.2:** Display prediction results alongside a comparison between BiLSTM and BERT outputs.
- **Activity 5.3:** Present the AI-generated guidance to the user within the interface.
- **Activity 5.4:** Build an analytics dashboard and a history page for reviewing past interactions.

Milestone 6: Analytics & Visualization

- **Activity 6.1:** Visualize emotion distribution across all logged interactions.
- **Activity 6.2:** Chart confidence scores and daily emotion trends.
- **Activity 6.3:** Compare BiLSTM and BERT model performance visually.

Milestone 7: CSV Logging

- **Activity 7.1:** Log the timestamp and raw user input for every interaction.
- **Activity 7.2:** Record the predicted emotion, confidence score, and any detected mixed emotions.
- **Activity 7.3:** Store the selected model and the corresponding Gemini AI response.

Milestone 8: Testing & Optimization

- **Activity 8.1:** Validate the text preprocessing pipeline.
- **Activity 8.2:** Evaluate the prediction accuracy of the emotion classification models.
- **Activity 8.3:** Test the Streamlit UI and verify that logging functions correctly.
- **Activity 8.4:** Perform performance optimization and final deployment readiness checks.

Milestone 1: Environment Setup & Dependency Configuration

This milestone focuses on preparing the development environment and installing all libraries and dependencies required by the AI Learning Assistant.

- **Activity 1.1:** Install Python 3.8+ and required system tools before beginning development.
- **Activity 1.2:** Install core dependencies: Streamlit, TensorFlow/Keras, PyTorch, Hugging Face Transformers, NLTK, Plotly, Pandas, NumPy, and Scikit-learn.
- **Activity 1.3:** Configure the Gemini API key securely using a .env file.

Milestone 2: Kaggle Model Training & Integration

This milestone covers training the BiLSTM and BERT emotion-classification models on Kaggle GPU using an emotion dataset, including preprocessing, tokenization, label encoding, evaluation, and exporting the trained models.

- **Activity 2.1:** Perform preprocessing, tokenization, and label encoding on the emotion training dataset.
- **Activity 2.2:** Train the BiLSTM and BERT models on Kaggle GPU resources.
- **Activity 2.3:** Evaluate model performance and export the trained models for integration.

Milestone 3: Emotion Detection Pipeline

This milestone builds the core pipeline that preprocesses text, tokenizes it, classifies emotions using BiLSTM and BERT, computes confidence scores, detects mixed emotions, and determines the final prediction.

- **Activity 3.1:** Preprocess and tokenize incoming student text input.
- **Activity 3.2:** Classify emotions using both the BiLSTM and BERT models.
- **Activity 3.3:** Compute confidence scores for each prediction.
- **Activity 3.4:** Detect mixed emotions and determine the final predicted emotion.

Milestone 4: AI-Powered Guidance (Gemini AI Integration)

This milestone uses prompt engineering to generate empathetic explanations, study strategies, motivation, and next steps based on the emotion detected in the pipeline.

- **Activity 4.1:** Design emotion-aware prompts using prompt engineering techniques.
- **Activity 4.2:** Generate empathetic explanations, personalized study strategies, motivation, and next steps via Gemini AI based on the detected emotion.

Milestone 5: Streamlit UI Development

This milestone delivers the Streamlit interface, providing a problem input form, prediction results, model comparison, AI guidance, an analytics dashboard, and a history page.

- **Activity 5.1:** Build a problem input form for students to describe their learning challenges.
- **Activity 5.2:** Display prediction results alongside a comparison between BiLSTM and BERT outputs.
- **Activity 5.3:** Present the AI-generated guidance to the user within the interface.
- **Activity 5.4:** Build an analytics dashboard and a history page for reviewing past interactions.

Milestone 6: Analytics & Visualization

This milestone uses Plotly to display emotion distribution, confidence scores, daily trends, and model comparison charts.

- **Activity 6.1:** Visualize emotion distribution across all logged interactions.
- **Activity 6.2:** Chart confidence scores and daily emotion trends.
- **Activity 6.3:** Compare BiLSTM and BERT model performance visually.

Milestone 7: CSV Logging

This milestone stores the timestamp, user input, predicted emotion, confidence score, mixed emotions, selected model, and Gemini response for every interaction.

- **Activity 7.1:** Log the timestamp and raw user input for every interaction.
- **Activity 7.2:** Record the predicted emotion, confidence score, and any detected mixed emotions.
- **Activity 7.3:** Store the selected model and the corresponding Gemini AI response.

Milestone 8: Testing & Optimization

This milestone performs preprocessing validation, prediction accuracy evaluation, UI testing, logging verification, performance optimization, and deployment readiness checks.

- **Activity 8.1:** Validate the text preprocessing pipeline.
- **Activity 8.2:** Evaluate the prediction accuracy of the emotion classification models.
- **Activity 8.3:** Test the Streamlit UI and verify that logging functions correctly.
- **Activity 8.4:** Perform performance optimization and final deployment readiness checks.

Hardware & Software Requirements:

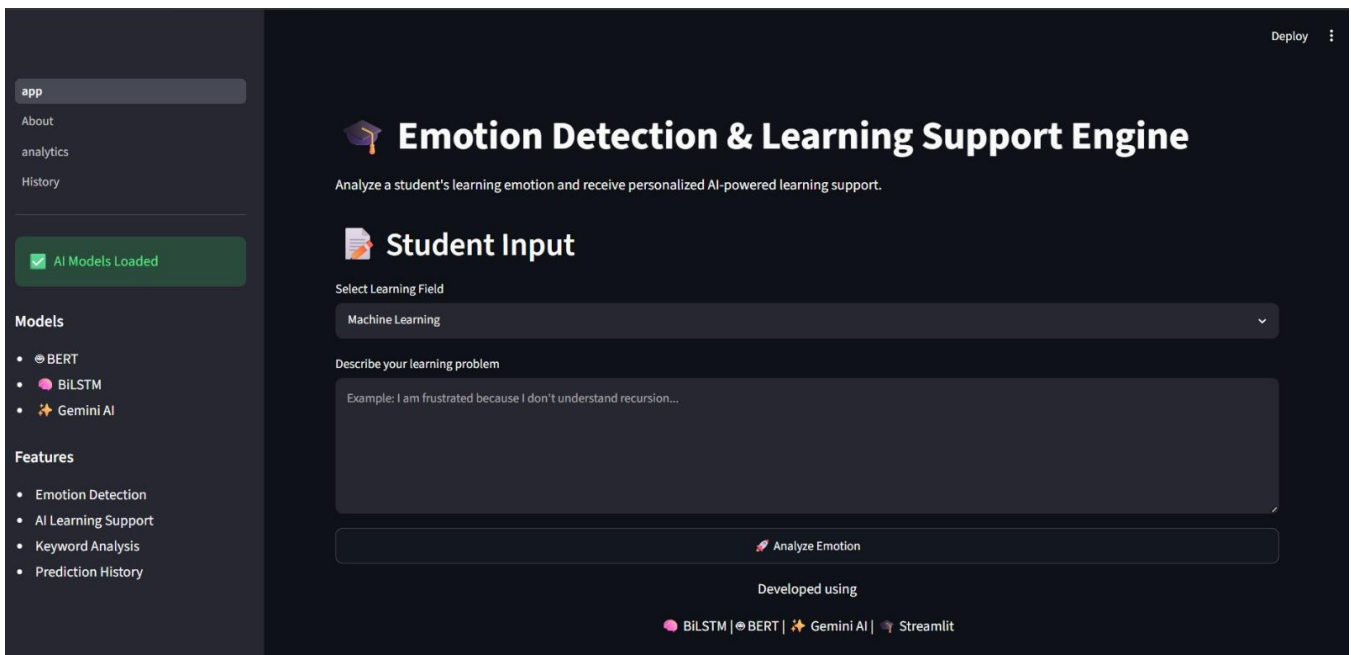
- Processor: Intel i3 or above
- RAM: 4 GB minimum (8 GB recommended)
- Storage: 2 GB free
- Operating System: Windows / Linux / macOS
- Python 3.8+
- Jupyter Notebook
- VS Code or PyCharm
- Internet connection

Future Scope:

- Voice emotion detection.
- Multilingual support.
- Adaptive learning.
- Mobile application.
- Cloud database.
- Educator dashboards.
- Reinforcement learning-based personalization.

Exploring the Website's Web Pages:

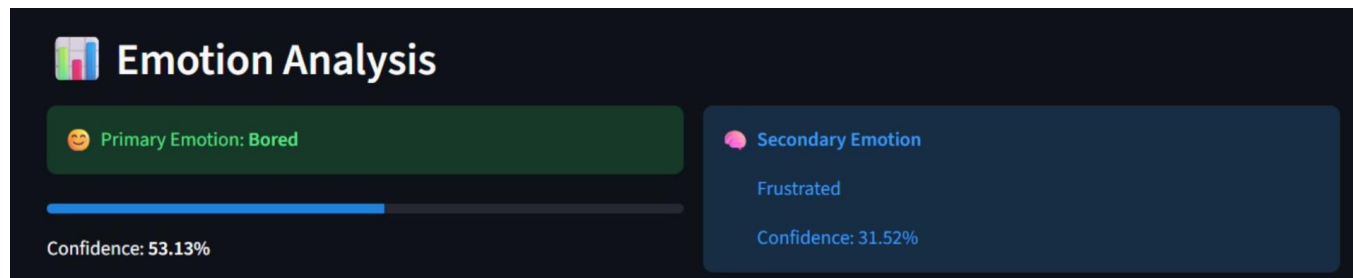
Home Page:



The screenshot shows the home page of the "Emotion Detection & Learning Support Engine". The interface has a dark theme. On the left is a sidebar with navigation links: "app" (highlighted), "About", "analytics", and "History". Below these is a green status bar that says "AI Models Loaded". Under "Models", there are three options: "BERT" (selected with a radio button), "BiLSTM", and "Gemini AI". Under "Features", there is a list: "Emotion Detection", "AI Learning Support", "Keyword Analysis", and "Prediction History". The main content area has a header "Emotion Detection & Learning Support Engine" with a graduation cap icon. Below it is a subtitle "Analyze a student's learning emotion and receive personalized AI-powered learning support." The "Student Input" section includes a "Select Learning Field" dropdown menu currently set to "Machine Learning". Below this is a text area for "Describe your learning problem" with an example: "Example: I am frustrated because I don't understand recursion...". At the bottom of the main area is a large "Analyze Emotion" button. The footer mentions "Developed using" and lists the technologies: "BiLSTM | BERT | Gemini AI | Streamlit". A "Deploy" button is visible in the top right corner.

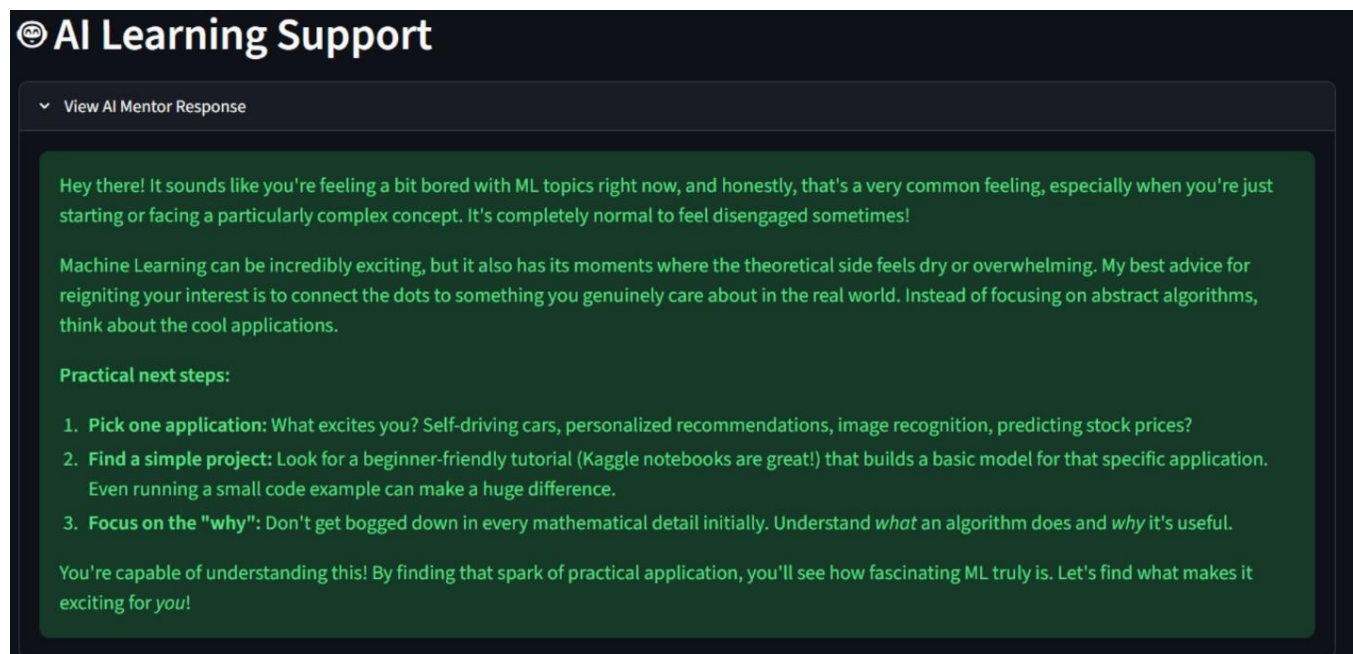
Description: The interface of Emotion Detection & Learning Support Engine serves as both the landing page and the generation tool. It features a dark background design with minimal interface where you can enter an input and analyze the emotion of the input text.

Result Page:



Description: This section displays the output for the give input text in the form of the emotion analysis and it displays the Primary Emotion and its percentage, Secondary Emotion and its percentage. For example the emotions detected here are Bored and Frustrated with respective percentages of 53.13% and 31.52%.

Learning Support:



AI Learning Support

View AI Mentor Response

Hey there! It sounds like you're feeling a bit bored with ML topics right now, and honestly, that's a very common feeling, especially when you're just starting or facing a particularly complex concept. It's completely normal to feel disengaged sometimes!

Machine Learning can be incredibly exciting, but it also has its moments where the theoretical side feels dry or overwhelming. My best advice for reigniting your interest is to connect the dots to something you genuinely care about in the real world. Instead of focusing on abstract algorithms, think about the cool applications.

Practical next steps:

- Pick one application:** What excites you? Self-driving cars, personalized recommendations, image recognition, predicting stock prices?
- Find a simple project:** Look for a beginner-friendly tutorial (Kaggle notebooks are great!) that builds a basic model for that specific application. Even running a small code example can make a huge difference.
- Focus on the "why":** Don't get bogged down in every mathematical detail initially. Understand *what* an algorithm does and *why* it's useful.

You're capable of understanding this! By finding that spark of practical application, you'll see how fascinating ML truly is. Let's find what makes it exciting for *you*!

Description: This section displays the AI Mentor's personalized response based on the student's detected emotional state and learning challenge. It provides empathetic encouragement, practical study strategies, and actionable next steps to help learners overcome difficulties and stay motivated. The guidance is tailored using Gemini AI, ensuring supportive, context-aware, and emotion-focused learning assistance.

Conclusion:

The AI Learning Assistant combines NLP, deep learning, generative AI, and interactive analytics into an end-to-end platform for emotion-aware learning support. It enables rapid identification of learner emotions, provides personalized academic guidance, and supports educators with transparent AI-assisted decision making.